

环境：ROS + Gazebo 课程资料：robot_soccer_integrated_demo

基础作业

1.1 Demo 复现

运行综合实践 Demo，观察 Gazebo 足球场环境、机器人、足球、球门、相机图像、感知结果和决策输出。

先启动基础环境：

```
roslaunch robot_soccer_integrated_demo soccer_env.launch
```

然后在新终端中选择 **Hough Demo** 或 **YOLO Demo** 其中一个运行。

如果 YOLO 环境或运行速度不稳定，可以优先选择 Hough Demo 完成基础作业。

Hough Demo：

```
roslaunch robot_soccer_integrated_demo soccer_hough_demo.launch
```

YOLO Demo：

```
roslaunch robot_soccer_integrated_demo soccer_yolo_demo.launch
```

可以使用以下命令查看关键输出：

```
rqt_image_view
```

```
rostopic echo /robot1/decision_state
```

```
rostopic echo /robot1/cmd_vel
```

如果使用 rqt_image_view，可选择查看：

```
/robot1/camera/image_raw
```

```
/robot1/hough/debug_image
```

```
/robot1/yolo/debug_image
```

提交内容：

- Gazebo 足球场运行截图；
- 机器人、足球和球门可见的截图；
- 相机画面或 debug 图像截图；
- /robot1/decision_state 输出截图；
- /robot1/cmd_vel 输出截图；

- 简要说明 Demo 中机器人完成了哪些动作或状态变化。

1.2 ROS 通信流程图

根据课堂 Demo 和运行结果，梳理足球机器人综合实践中的 ROS 通信流程，画出从图像输入到机器人速度输出的流程图。

流程图至少包含以下内容：

Gazebo 足球场

/robot1/camera/image_raw

/robot1/vision/detections

/robot1/ball_pose

/robot1/goal_pose

/robot1/decision_state

/robot1/cmd_vel

可以参考以下主线：

Gazebo 场景 ↓ 机器人相机图像 ↓ Hough / YOLO 足球检测 ↓ 视觉检测结果
 ↓ 球位置估计 ball_pose ↓ FSM 决策 ↓ cmd_vel 速度指令 ↓ 机器人运动

要求：

- 画出一张 ROS 节点与话题通信流程图；
- 整理一个话题说明表；
- 用自己的话说明机器人如何从“看到球”变成“输出速度指令”。

请将基础作业整理为 **一份 PDF** 提交，包含：

- Demo 运行截图；
- 相机画面或 debug 图像截图；
- /robot1/decision_state 和 /robot1/cmd_vel 输出截图；
- ROS 节点与话题通信流程图；
- 话题说明表；
- 从感知到决策再到控制的简要说明。

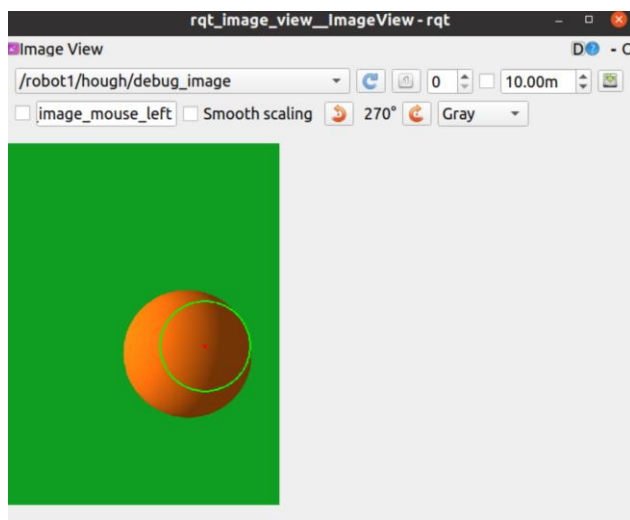
Answers:

1.1 Demo 复现

(1) Gazebo 足球场、机器人、足球和球门可见的截图



(2) Hough debug 图像截图



(3) /robot1/decision_state 和 /robot1/cmd_vel 输出截图

```

andrew@vm1:~/code/robot-course-practice/myspace/15/catkin_ws$ rostopic echo /robot1/decision_state
data: "DRIBBLE_BALL"
---
data: "DRIBBLE_BALL"
---
data: "DRIBBLE_BALL"
---
data: "DRIBBLE_BALL"
---
data: "DRIBBLE_BALL"
---
data: "DRIBBLE_BALL"
---
data: "DRIBBLE_BALL"
---
data: "DRIBBLE_BALL"
---
data: "DRIBBLE_BALL"
---
data: "DRIBBLE_BALL"
---
data: "DRIBBLE_BALL"
---
data: "DRIBBLE_BALL"
---
data: "DRIBBLE_BALL"
---
data: "DRIBBLE_BALL"
---
data: "DRIBBLE_BALL"
---
data: "SHOOT"
---
data: "SHOOT"
---
data: "DRIBBLE_BALL"
---
data: "DRIBBLE_BALL"

```

```

andrew@vm1:~/code/robot-course-practice/myspace/15/catkin_ws$ rostopic echo /robot1/cmd_vel
z: 0.0
angular:
x: 0.0
y: 0.0
z: -0.002476431479681844
---
linear:
x: 0.09
y: 0.0
z: 0.0
angular:
x: 0.0
y: 0.0
z: -0.0013795679247854433
---
linear:
x: 0.09
y: 0.0
z: 0.0
angular:
x: 0.0
y: 0.0
z: -0.00024517211644491723

```

(4) Demo 中机器人完成的动作或状态变化

1. 分析如下 script 的基本功能

- hough_circle_detector_node.py

订阅 camera/image_raw，回调_image_callback，发布检测到的最佳圆(x,y,radius)到 /robot1/vision/detections。

- vision_pose_provider.py

订阅 /robot1/vision/detections，回调_detection_callback，赋值 self.detection 和 self.detection_time。循环中向/robot1/goal_pose 推送球门相对机器人坐标系位姿，向 /robot1/ball_pose 推送球相对机器人坐标系位姿。

- soccer_demo_node.py

订阅/robot1/ball_pose，回调_ball_callback，赋值 self.ball_pose 和 self.ball_time。订阅 /robot1/goal_pose，回调_goal_callback，赋值 self.goal_pose 和 self.goal_time。循环中

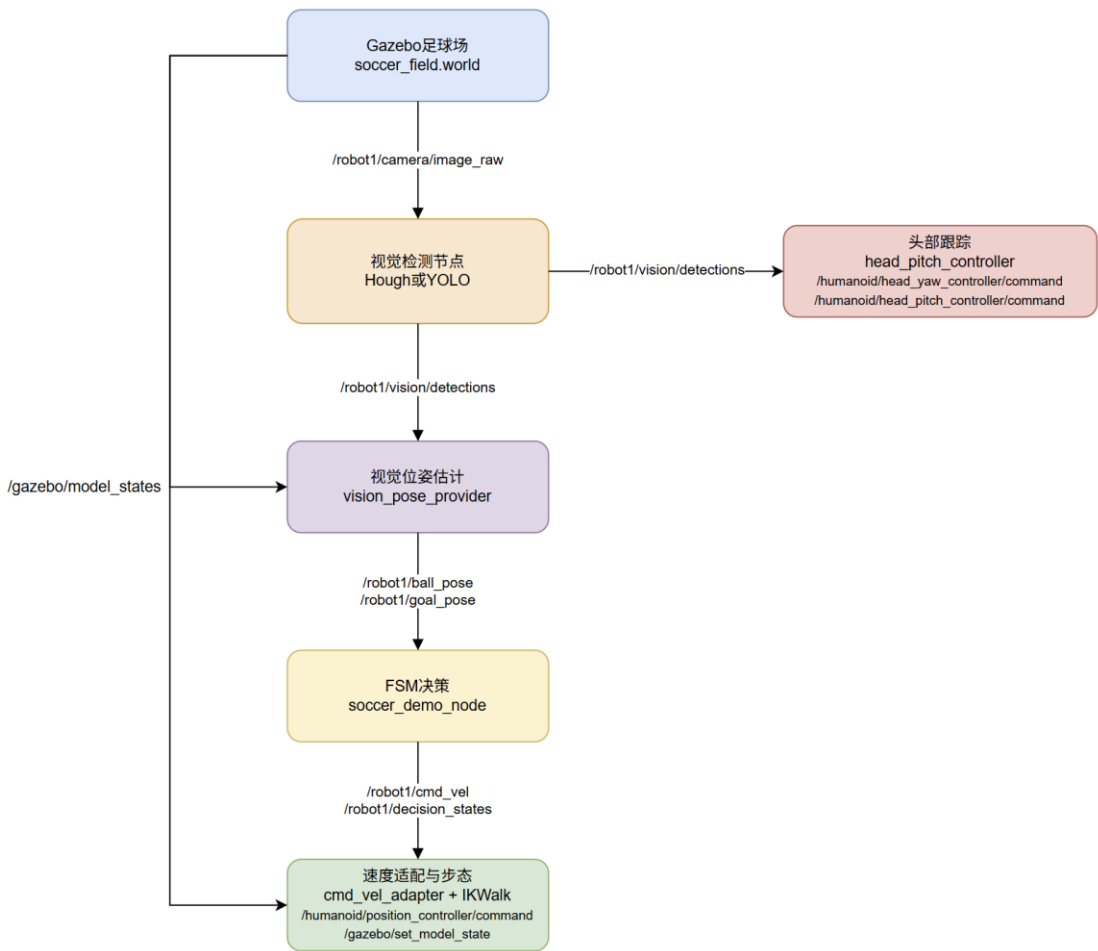
计算 distance，进入 FSM 向/robot1/cmd_vel 和/robot1/decision_state 推送关节指令和状态。

2. 机器人动作和状态变化简要说明

机器人首先在足球场中保持站立，当视觉节点尚未检测到足球时，决策状态为 **SEARCH_BALL**，机器人以较小角速度原地搜索。当 Hough 圆检测或 YOLO 检测在相机画面中识别到足球后，头部跟踪节点根据检测结果调整，使足球尽量保持在视野中心。决策节点 soccer_demo_node 根据球的相对距离和方位切换状态：距离较远时进入 **APPROACH_BALL**，发布前进速度并叠加转向；距离进入带球范围后进入 **DRIBBLE_BALL**，降低前进速度并继续修正方向；当足球足够近时进入 **SHOOT**，朝球门方向输出更大的前进速度完成射门动作。如果短时间丢失足球，状态会变为 **RECOVER**，机器人暂停明显运动等待检测恢复；超过恢复时间仍未看到球，则重新进入 SEARCH_BALL。

1.2 ROS 通信流程图

(1) ROS 节点与话题通信流程图



(2) 话题说明表

话题	说明
/robot1/camera/image_raw	消息类型: sensor_msgs/Image 发布者: Gazebo 相机插件 订阅者: hough_circle_detector_node 作用: 足球检测的输入
/robot1/vision/detections	消息类型: std_msgs/Float32MultiArray 发布者: Hough 检测节点 订阅者: head_pitch_controller、vision_pose_provider 作用: 检测结果, 字段为 x_error、y_error、宽高归一化、置信度、类别
/robot1/hough/debug_image	消息类型: sensor_msgs/Image 发布者: 对应检测节点 订阅者: rqt_image_view 作用: 显示检测框、圆心, 用于确认足球是否被识别
/humanoid/head_yaw_controller/command、 /humanoid/head_pitch_controller/command	消息类型: std_msgs/Float64 发布者: head_pitch_controller 订阅者: Gazebo 头部关节控制器 作用: 使足球保持在视野中心
/gazebo/model_states	消息类型: gazebo_msgs/ModelStates 发布者: Gazebo 订阅者: vision_pose_provider、cmd_vel_adapter 作用: 提供机器人和足球等模型的世界位姿
/robot1/ball_pose	消息类型: geometry_msgs/PoseStamped 发布者: vision_pose_provider 订阅者: soccer_demo_node 作用: 发布足球在 robot1/base_link 坐标系下的相对位置
/robot1/goal_pose	消息类型: geometry_msgs/PoseStamped 发布者: vision_pose_provider 作用: 朝向修正
/robot1/decision_state	消息类型: std_msgs/String 发布者: soccer_demo_node 订阅者: 调试观察 作用: 输出 FSM 状态
/robot1/cmd_vel	消息类型: geometry_msgs/Twist 发布者: soccer_demo_node 订阅者: cmd_vel_adapter 作用: 发布线速度和角速度
/humanoid/position_controller/command	消息类型: std_msgs/Float64MultiArray 发布者: cmd_vel_adapter 订阅者: Gazebo 位置控制器 作用: 将 cmd_vel 转换为 IKWalk 关节命令
/robot1/reset_ball、/robot1/reset_demo	消息类型: std_srvs/Empty 发布者: vision_pose_provider 订阅者: 人工调用 作用: 进球或测试后复位足球, 或同时复位机器人和足球

(3) 从感知到决策再到控制的简要说明

首先, Gazebo 相机持续发布/robot1/camera/image_raw。Hough Demo 使用 HSV 阈值和 HoughCircles 检测橙色足球并把结果整理成/robot1/vision/detections 格式。

其次, head_pitch_controller 直接订阅 detections 中的 x_error 和 y_error, 让头部 yaw/pitch

朝检测目标修正； vision_pose_provider 同时读取 detections、头部关节状态和 /gazebo/model_states，把图像坐标误差换算成足球相对机器人底座的 ball_pose，并计算球门相对位置 goal_pose。

最后， soccer_demo_node 订阅 ball_pose 和 goal_pose 并执行 FSM：看不到球时搜索，看见球且较远时靠近，接近后低速带球，足够近时朝球门射门。每次状态更新都会同步发布 /robot1/decision_state 和 /robot1/cmd_vel； cmd_vel_adapter 再把 Twist 速度转成 IKWalk 关节命令与 Gazebo 模型位姿更新，从而让机器人在仿真中移动。